

Algo ZH II.

Tartalom

Fák, bináris fák.....	1
Bináris fák mérete és magassága.....	3
Bináris fák láncolt ábrázolásai	3
Bináris fák bejárásai (tree traversals)	4
Bináris fák zárójelezett, szöveges formája	5
Bináris keresőfák	6
Keresőfa:.....	6
Rendezőfa:	6
A bináris keresőfák műveletei:	7
Kupac	8
Kupac ábrázolása	9
Elsőbbségi (prioritásos sorok)	9
Kupac műveletek	10
Beszúrás.....	10
Maximális elem törlése	10
Rendezés elsőbbségi sorral	11
Kupacrendezés (heap sort).....	11
Működése	11

Megjegyzés: Az előadásos pdfben nagyon sok olyan stuktogram/lejátszás/hasznos dolog van, amit én ide nem illesztettem be, érdemes rájuk nézni!!

Fák, bináris fák

A (bináris) fákat nagy méretű adathalmazok és multihalmazok (zsákok) ábrázolására, de egyéb adatrepresentációs célokra is gyakran használjuk.

A bináris fák esetében minden adatelemnek vagy szokásos nevén csúcsnak (angolul node) legfeljebb kettő rákövetkezője van: egy bal (left) és/vagy egy jobb (right) rákövetkezője. -> Ezeket a csúcs gyerekeinek (children) nevezzük

A **csúcs** a gyerekei **szülője** (parent),

Az azonos szülőjű csúcsok pedig egymás **testvérei** (siblings)

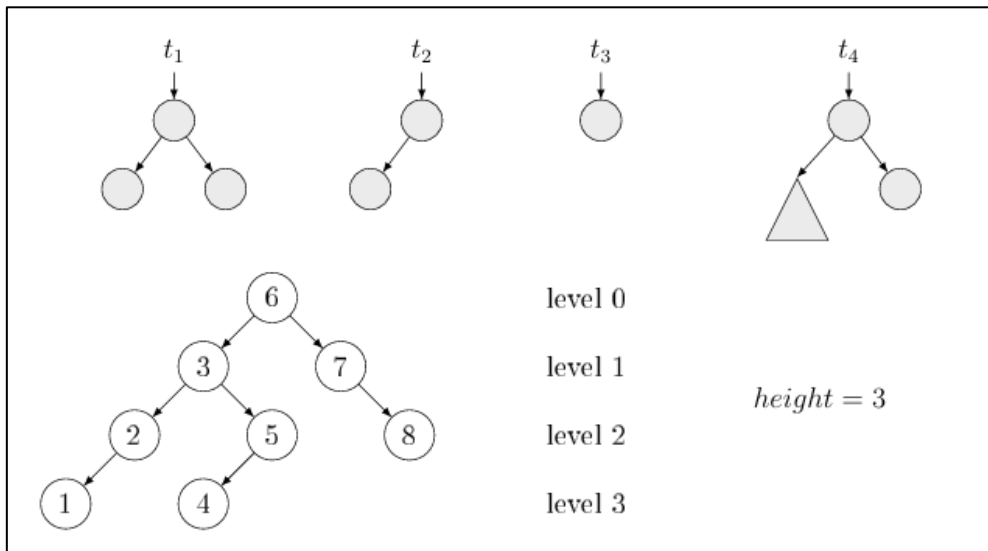
Levél (leaf): olyan csúcs, aminek nincs gyereke

Gyökér (root) csúcs: olyan csúcs, aminek nincs szülője

Belső csúcs (internal node): nem-levél csúcs

A fában egy csúcs **leszármazottai** (descendants) a gyerekei és a gyerekei leszármazottai.

Egy csúcs **ősei** (ancestors) a szülője és a szülője ősei.



Listává torzult, szigorúan bináris, tökéletes és majdnem teljes bináris fák

Listává torzult fa: Azon fák, amelyekben minden belső (azaz nem-levél) csúcsnak egy gyereke van

Szigorúan bináris fa: Azon fák, amelyekben minden belső (azaz nem-levél) csúcsnak két gyereke van

Tökéletes bináris fa: Olyan szigorúan bináris fa, amelynek minden levele azonos szinten van (Ilyenkor az összes levél szükségszerűen a fa legmélyebb szintjén található, a felsőbb szinteken levő csúcsok pedig belső csúcsok, így két-két gyerekük van.)

- h magasságú tökéletes bináris fa csúcsainak száma: $1 + 2 + 4 + \dots + 2^h = 2^{h+1} - 1$

Majdnem teljes bináris fa: Ha egy tökéletes bináris fa levélszintjéről nulla, egy vagy több levelet elveszünk, de nem az összeset

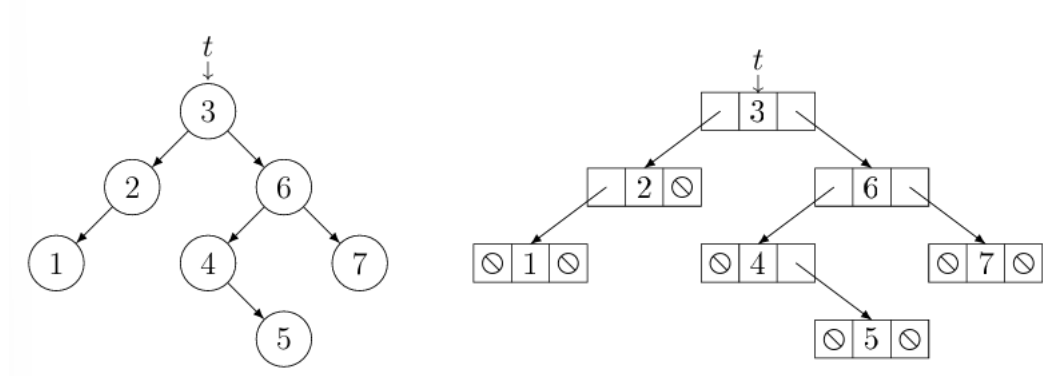
- az üres fát is majdnem teljesnek tekintjük
- minden tökéletes bináris fa egyben majdnem teljes is

Bináris fák mérete és magassága

- Bináris fa mérete: csúcsainak száma
- A t bináris fa mérete: $|t|$ vagy $n(t)$ ((vagy simán n , ha egyértelműen tudjuk melyik fáról van szó))
- A gyökér van a nulladik szinten
- Az i -edik szintű csúcsok gyerekeit az $(i+1)$ -edik szinten találjuk
- A fa magassága: a legmélyebben fekvő levelei szintszáma
- A t bináris fa magassága: $h(t)$ ((vagy simán h , ha egyértelműen tudjuk melyik fáról van szó))
- Az üres fa magassága -1
- Magasság tetszőleg nemüres fára:

$$h(t) = 1 + \max(h(t \rightarrow \text{left}), h(t \rightarrow \text{right}))$$

Bináris fák láncolt ábrázolásai



A bináris fa csúcsait pl. az alábbi Node osztály objektumaiként ábrázolhatjuk.

Tetszőleges, láncoltan ábrázolt bináris fát egy Node^* típusú pointer azonosít. Ez $\emptyset$, ha a fa üres. A gyökércsúcsára hivatkozik, ha a fa nemüres.

Néha hasznos, ha a csúcsokban van egy parent szülő pointer is, mert a fában így felfelé is tudunk haladni.

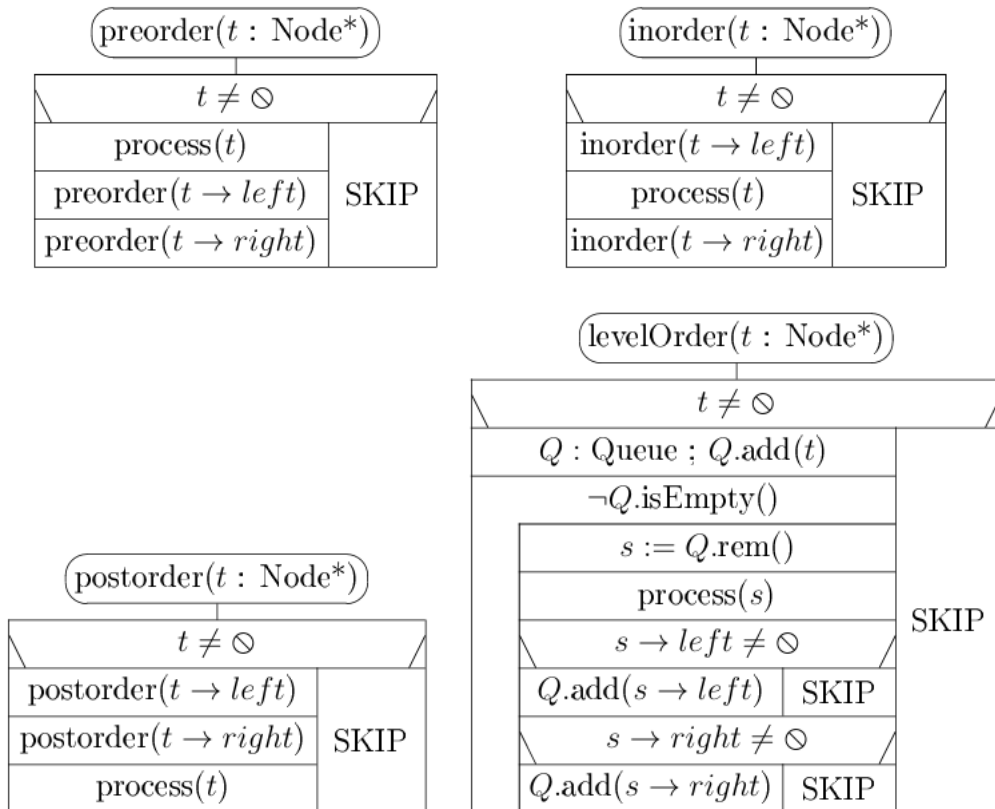
Node
+ $key : \mathcal{T} // \mathcal{T}$ valamilyen ismert típus
+ $left, right : \text{Node}^* := \emptyset$
+ $\text{Node}(x : \mathcal{T}) \{ key := x \} //$ egycsúcsú fát képez belőle

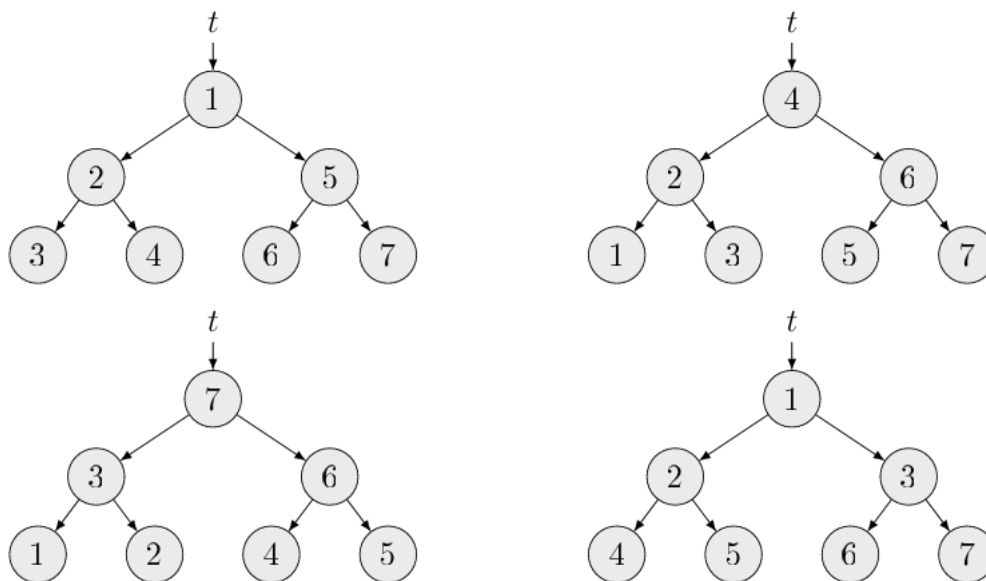
Node3
+ $key : \mathcal{T} // \mathcal{T}$ valamilyen ismert típus
+ $left, right, parent : Node3^* := \emptyset$
+ $Node3(x : \mathcal{T}) \{ key := x \}$

Bináris fák bejárásai (tree traversals)

Üres fára mindegyik bejárás az üres program. Nemüres r-áris fákra:

- **Preorder:** először a fa gyökerét dolgozza fel, majd sorban bejárja a $0..r-1$ -edik részfákat
- **Postorder:** előbb sorban bejárja a $0..r-1$ -edik részfákat, és a fa gyökerét csak a részfák után dolgozza fel
- **Inorder:** először bejárja a nulladik részfát, ezután a fa gyökerét dolgozza fel, majd sorban bejárja az $1..r-1$ -edik részfákat
- **Szintenkénti bejárás:** a csúcsokat a gyökértől kezdve szintenként, minden szintet balról jobbra bejárva dolgozza fel





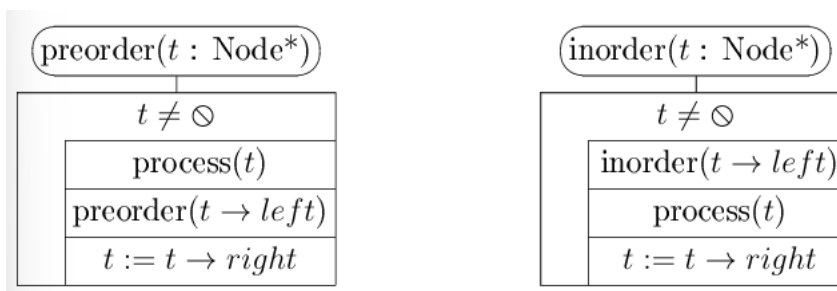
17. ábra. Bal felső sarokban preorder, jobb felsőben inorder, bal alsóban postorder, jobb alsóban szintenkénti bejárása látható a t bináris fának.

s

Állítás: $T_{\text{preorder}}(n), T_{\text{inorder}}(n), T_{\text{postorder}}(n), T_{\text{levelOrder}}(n) \in \Theta(n)$, ahol n a fa mérete, azaz csúcsainak száma.

Igazolás: Az első három bejárás pontosan annyiszor hívódik meg, amennyi részfája van az eredeti bináris fának (az üres részfákat is beleszámolva), és egy-egy hívás végrehajtása $\Theta(1)$ futási időt igényel. Másrészt n szerinti teljes indukcióval könnyen belátható, hogy tetszőleges n csúcsú bináris fának $2n + 1$ részfája van. A szintenkénti bejárás pedig mindegyik csúcsot a ciklus egy-egy végrehajtásával dolgozza fel.

A preorder és az inorder bejárások hatékonysága konstans szorzóval javítható, ha a végrekurziókat ciklussá alakítjuk



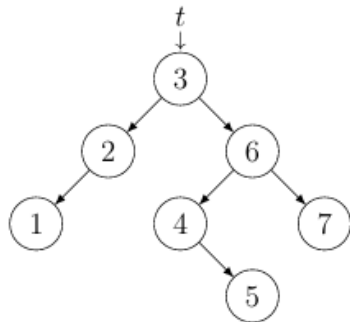
Bináris fák zárójelezett, szöveges formája

Tetszőleges nemüres bináris fa zárójeles, azaz szöveges alakja (textual form):

(balRészFa Gyökér jobbRészFa)

- Az üres (rész)fát egy üres zárójelpár reprezentálja. pl. ()

- A könnyebb olvashatóság kedvéért a levélcsúcsok üres részfáihoz tartozó üres zárójelpárokat elhagyjuk
 - o Pl. az 5 kulcsú, egy csúcsú bináris fa szöveges reprezentációja $(() (5) ())$ helyett egyszerűen (5)
- A zárójeleket elhagyva, a fa inorder bejárását kapjuk.
- Elegáns zárójeles alak: több féle zárójel (szinttől függően)



A balra látható t bináris fa

- egyszerű zárójeles alakja:
 $(((1) 2 ()) 3 ((() 4 (5)) 6 (7)))$
- elegáns zárójeles alakja:
 $\{ [(1) 2 ()] 3 [(() 4 \langle 5 \rangle) 6 (7)] \}$

Bináris keresőfák

Keresőfa:

Egy bináris fát keresőfának nevezünk, ha minden belső csúcsára és annak y kulcsára igazak az alábbi követelmények:

- A csúcs bal részfájában tetszőleges csúcs x kulcsára $x < y$
- A csúcs jobb részfájában tetszőleges csúcs z kulcsára $z > y$

Rendezőfa:

Egy bináris fát rendezőfának (binary sort tree) nevezünk, ha minden belső csúcsára és annak y kulcsára igazak az alábbi követelmények:

- A csúcs bal részfájában tetszőleges csúcs x kulcsára $x \leq y$.
- A csúcs jobb részfájában tetszőleges csúcs z kulcsára $z \geq y$.

- A keresőfában tehát minden kulcs egyedi, míg a rendezőfában lehetnek duplikált és többszörös kulcsok is.
- A bináris keresőfák tekinthetők véges halmazok, vagy szigorúan monoton növekvő sorozatok reprezentációinak

Jelölje $H(t)$ a t bináris keresőfa által reprezentált halmazt! Ekkor definíció szerint $H(\emptyset) = \{\}$, illetve $H(t) = H(t \rightarrow \text{left}) \cup \{t \rightarrow \text{key}\} \cup H(t \rightarrow \text{right})$, amennyiben $t \neq \emptyset$.

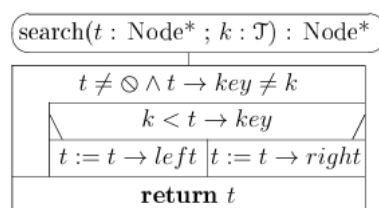
- A továbbiakban feltesszük, hogy a bináris keresőfákat láncoltan ábrázoljuk, szülő pointerrek nélkül, azaz a fák csúcsai Node típusúak, és az üres fát a null pointer reprezentálja

A bináris keresőfák műveletei:

Mindegyik műveletre $MT(h) \in \mathcal{O}(h)$ (ahol $h = h(t)$)

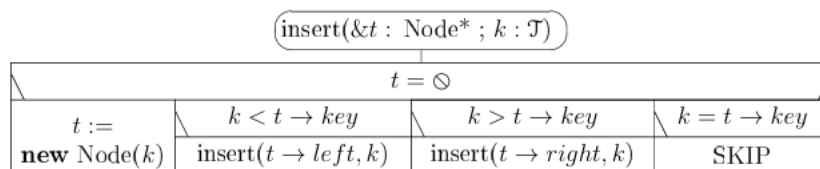
- **search(t,k) függvény** : ha $k \in H(t)$, akkor a k kulcsú csúcsra mutató pointerrel tér vissza, különben a hivatkozással

A search(t,k) függvény a t fában, a bináris keresőfa definíciója alapján meg keresi a k kulcs helyét. A kulcsot akkor és csak akkor találja meg, ha ott nemüres részfa van



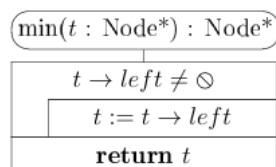
- **insert(t, k) : a $H(t) := H(t) \cup \{k\}$ absztrakt művelet megvalósítása**

Az insert(t,k) eljárás is megkeresi a t fában a k kulcs helyét. Ha ott egy üres részfat talál, akkor az üres részfa helyére tesz egy új levélcsúcsot, k kulccsal



- **min(t) függvény** : a t nemüres fában a minimális kulcsú csúcs címét adja vissza, pontosabban, az inorder bejárás szerinti első csúcsát

A min(t) függvény a t nemüres fa "bal alsó" csúcsára hivatkozó pointerrel tér vissza.



- **remMin(t,minp)** : a t nemüres fából kivesszük a min(t) függvény által meghatározott csúcsot, és a címét a minp pointerben adjuk vissza.

A remMin(t,minp) eljárás minp-ben a t nemüres fa "bal alsó" (a legkisebb kulcsot tartalmazó) csúcsára mutató pointerrel tér vissza, de még előtte a csúcsot kifűzi a fából, azaz a csúcshoz tartozó részfa helyére teszi a csúcs jobb oldali részfáját

remMin(&t, &minp : Node*)	
$t \rightarrow left = \emptyset$	
$minp := t$	remMin($t \rightarrow left$, minp)
$t := minp \rightarrow right$	
$minp \rightarrow right := \emptyset$	

- **del(t, k)** : a $H(t) := H(t) \setminus \{k\}$ absztrakt művelet megvalósítása

A del(t,k) eljárás szintén megkeresi a t fában a k kulcs helyét. Ha megtalálta a k kulcsot tartalmazó csúcsot, még két eset lehet.

- (1) Ha a csúcs egyik részfája üres, akkor a csúcshoz tartozó részfa helyére teszi a csúcs másik részfáját.
- (2) Ha a csúcsnak két gyereke van, akkor a remMin(t,minp) eljárás segítségével kiveszi a jobb oldali részfából a minimális kulcsú csúcsot, és a k kulcsú csúcs helyére teszi, hiszen ez a kulcs a bal oldali részfa kulcsainál nagyobb, a jobb oldali részfa maradék kulcsainál pedig kisebb

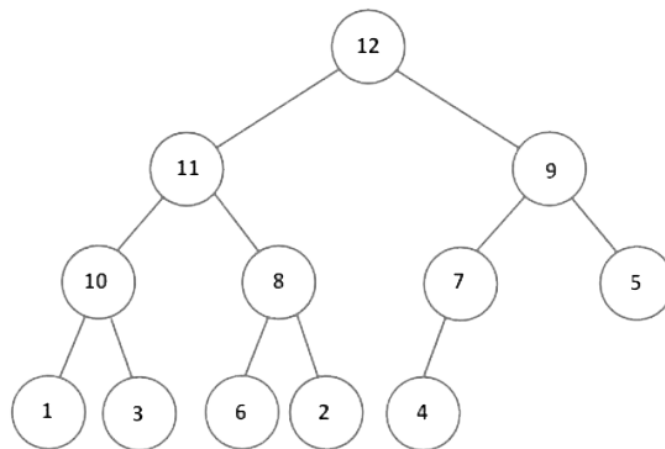
del(&t : Node* ; k : T)			
$t \neq \emptyset$			
$k < t \rightarrow key$	$k > t \rightarrow key$	$k = t \rightarrow key$	SKIP
del($t \rightarrow left$, k)	del($t \rightarrow right$, k)	delRoot(t)	

delRoot(&t : Node*)		
$p := t$		
$t \rightarrow left = \emptyset$	$t \rightarrow right = \emptyset$	$t \rightarrow left \neq \emptyset \wedge t \rightarrow right \neq \emptyset$
$t := p \rightarrow right$	$t := p \rightarrow left$	remMin($t \rightarrow right, q$)
		$q \rightarrow left := p \rightarrow left$
		$q \rightarrow right := p \rightarrow right$
		$t := q$
delete p		

Kupac

Egy teljes bináris fát maximum-kupacnak (heap) nevezünk, ha minden belső csúcs kulcsa nagyobb-egyenlő, mint a gyerekeié. Ha minden belső csúcs kulcsa kisebb-egyenlő, mint a gyerekeié, minimum-kupacról beszélünk. Ebben a félévben kupac alatt maximum-kupacot értünk.

A **maximum kupac** egy majdnem teljes, balra tömörített bináris fa, melynek minden belső csúcsára teljesül, hogy a belső csúcs kulcsa nagyobb vagy egyenlő a gyerekei kulcsánál. Így kupac gyökerében mindig az egyik legnagyobb elem található.



Kupac ábrázolása

A teljes bináris fát, speciálisan a kupacokat, szokás szintfolytonosan egy tömbben ábrázolni, ami legyen az egyszerűség kedvéért az $A : T[m]$ tömb

- Ha a fának n csúcsa van, akkor az $n \leq m$ felételnek teljesülnie kell
- A csúcsokat szintfolytonosan a tömb első n elemében, az $A[0..n)$ résztömbben tároljuk
- $A[0]$ a fa gyökercsúcsa, feltéve, hogy $n > 0$ ($n = 0$ az üres fa esete)
- Az $A[i]$ csúcs gyerekei $A[\text{left}(i)]$ és $A[\text{right}(i)]$ (feltéve, hogy $\text{left}(i) < n$, $\text{right}(i) < n$)

A tömbben a szülő-gyerek és a gyerek-szülő kapcsolatokat az alábbi képletek adják:

- csúcs indexe: i
- csúcs bal gyereke ($\text{left}(i)$): $2*i+1$
- csúcs jobb gyereke ($\text{right}(i)$): $\text{left}(i)+1 = 2*i+2$
- csúcs szülője ($\text{parent}(i)$): Alsóegészrész $(i-1)/2$
- hányadik szinten van a csúcs: Alsóegészrész $\log_2(i+1)$

Elsőbbségi (prioritásos sorok)

Az elsőbbségi sor (priority queue) egy zsák (multihalmaz), amelybe be tudunk tenni újabb elemeket, és ki tudjuk választani, illetve kivenni az egyik maximális elemét.

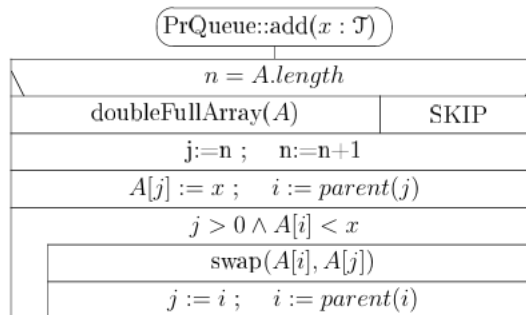
Az alábbiakban a prioritásos sor típust a PriorityQueue osztály segítségével írjuk le. Az elsőbbségi sor aktuális elemeit az $A[1..n]$ résztömb tartalmazza, ami egy kupac.

PrQueue
$- A : \mathcal{T}[]$ // $\mathcal{T}$ is some known type ; $A.length$ is the physical $- \text{constant } m0 : \mathbb{N}_+ := 16$ // size of the PrQ, its default is $m0$. $- n : \mathbb{N}$ // $n \in [0 \dots A.length]$ is the actual length of the PrQ
$+ \text{PrQueue}(m : \mathbb{N}_+ := m0) \{ A := \text{new } \mathcal{T}[m]; n := 0 \}$ // create an empty PrQ $+ \text{add}(x : \mathcal{T})$ // insert x into the priority queue $+ \text{remMax}() : \mathcal{T}$ // remove and return the maximal element of the priority queue $+ \text{max}() : \mathcal{T}$ // return the maximal element of the priority queue $+ \text{isEmpty}() : \mathbb{B} \{ \text{return } n = 0 \}$ $+ \sim \text{PrQueue}() \{ \text{delete } A \}$ $+ \text{setEmpty}() \{ n := 0 \}$ // reinitialize the priority queue

Kupac műveletek

Beszúrás

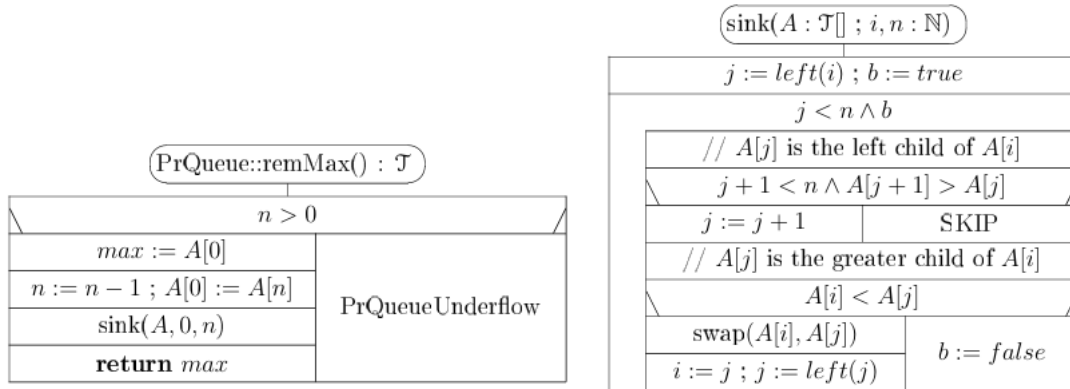
Az új elemet a tömbben az utolsó eleme után helyezzük el, így a kupac alakja továbbra is balra tömörített lesz. Ezután az új elemet addig emeljük a gyökér felé, amíg a kupac tulajdonság helyre nem áll, azaz helyet cserél a szülőjével mindaddig, míg a beszúrt elem kulcsa nagyobb, mint a szülőjének a kulcsa, vagy fel nem ér a kupac tetejére.



Maximális elem törlése

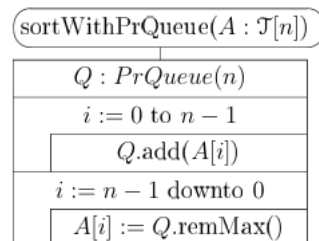
A $\text{remMax}()$ metódus a maximum elmentése után a kupac gyökerébe, $A[0]$ -ba teszi át a szintfolytonosan utolsó elemet. Ezzel a kupac mérete eggyel csökken, és a gyökerénél valószínűleg el is romlik (ún. csonka kupac lesz). Ezért az alábbi "sink" eljárás segítségével (amit mindig $i = 0$ -val hív meg) a gyökérbe átrakott elemet ($A[i]$) addig süllyeszti lefelé, amíg a helyére nem kerül.

Ennek során a lesüllyesztendő elemet mindig az aktuálisan nagyobb gyerekével cseréli meg, amíg még a levélszint fölött van, és a nagyobbik gyereke nagyobb nála. Ilyen módon a ciklus során a lesüllyedő elem mindig kisebb, mint az ősei. Amikor a ciklus megáll, akkor vagy leért a levélszintre, vagy $\geq$ mint a gyerekei, és kupac helyreáll



Rendezés elsőbbségi sorral

Könnyen és hatékonyan rendezhetünk tömböket



Kupacrendezés (heap sort)

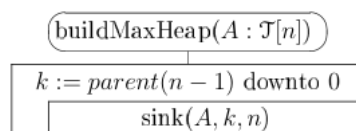
A kupacrendezés a fenti *sortWithPrQueue*($A : \mathcal{T}[]$) optimalizálása.

- A kupacrendezés során maximum kupacot használunk
- Helyben rendez
- Nem stabil
 - o a számokat az A tömbön belül rakja a helyes sorrendbe és legfeljebb csak állandó számú elem tárolódik a tömbön kívül
- Aszimptotikusan optimális összehasonlító rendezés, ugyanis a műveletigényének felső korlátja megegyezik az alsó korláttal ($O(n * \log n)$, $\Omega(n * \log n)$)

Működése

Először kupaccá alakítjuk a tömböt, meghívjuk a buildMaxHeap() eljárást.

Ez oly módon történik, hogy a legutolsó levél szülőjétől kezdve lesüllyesztjük a szülőket addig, amíg kisebb, mint a nagyobbik gyereke. Ezt az összes szülő csúcsra el kell végezni.



A kupaccá alakítás után tehát ezt ismételjük: Megcseréljük a kupac maximumát, azaz gyökerében lévő elemet ($A[0]$) a kupac szintfolytonosan utolsó elemével, miközben levágjuk a maximumot ($m := m-1$), majd lesüllyesztjük az $A[0..m)$ csonka kupacban a gyökérbe tett elemet. Így az előbbinél eggyel kisebb méretű kupacot kapunk ($A[0..m)$). Ezt a ciklusmagot addig ismételjük, amíg az $A[0..m)$ kupac mérete nagyobb, mint egy. Így az $A[0..n)$ tömbben visszafelé megkapjuk az első, második stb. maximumokat, és végül az $A[0]$ -ban a minimumot, azaz a tömb rendezve lesz.

